

Java 17 — Sealed Classes

Java Version: 17

Year: 2021

Feature: Sealed Classes — **Final**

UniqueClassName: SealedClasses

Definition

Sealed classes became a **final feature in Java 17**.

A sealed class or interface allows you to explicitly specify **which classes or interfaces are permitted to extend or implement it**.

It provides controlled inheritance.

Basic Syntax

```
public sealed class Shape
    permits Circle, Rectangle {
}
```

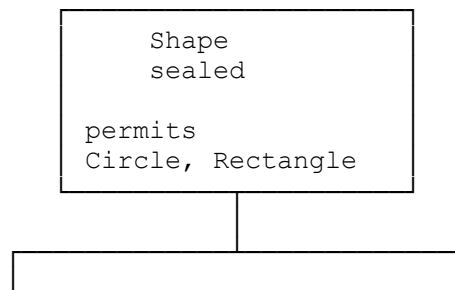
This means:

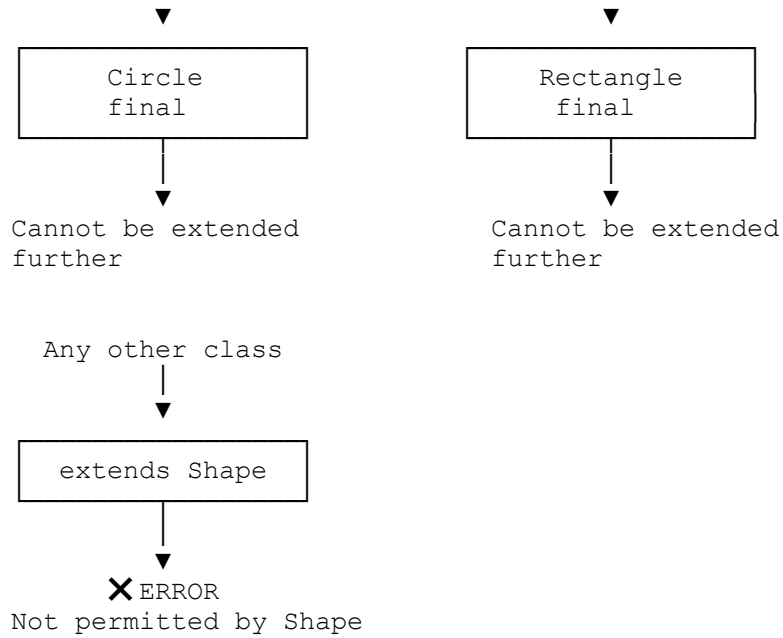
- Shape can be extended.
- Only Circle and Rectangle are permitted subclasses.
- Other classes cannot directly extend Shape.

The permitted subclasses must declare one of:

- final
- sealed
- non-sealed

Flowchart





Java Code Example

```
public class SealedClasses {

    // Sealed parent class
    public sealed abstract class Shape
        permits Circle, Rectangle {

        public abstract double area();
    }

    // Permitted subclass
    public final class Circle extends Shape {

        private final double radius;

        public Circle(double radius) {
            this.radius = radius;
        }

        @Override
        public double area() {
            return Math.PI * radius * radius;
        }
    }

    // Permitted subclass
    public final class Rectangle extends Shape {

        private final double length;
        private final double width;

        public Rectangle(double length, double width) {
```

```

        this.length = length;
        this.width = width;
    }

    @Override
    public double area() {
        return length * width;
    }
}

public static void main(String[] args) {

    SealedClasses app = new SealedClasses();

    Shape circle = app.new Circle(5);
    Shape rectangle = app.new Rectangle(10, 4);

    System.out.println("Circle area      : " + circle.area());
    System.out.println("Rectangle area : " + rectangle.area());
}
}

```

Output

```

Circle area      : 78.53981633974483
Rectangle area   : 40.0

```

final, sealed, and non-sealed

A permitted subclass has three choices.

1. final

```

public final class Circle extends Shape {
}

```

No class can extend Circle.

```

Shape
├── Circle (final)
│   └── ✗ Cannot extend

```

2. sealed

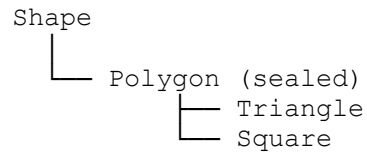
```

public sealed class Polygon
    extends Shape

```

```
} permits Triangle, Square {  
}
```

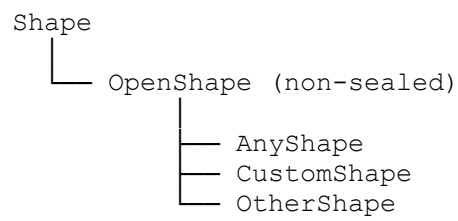
Polygon restricts its own subclasses.



3. non-sealed

```
public non-sealed class OpenShape extends Shape {  
}
```

The inheritance restriction is opened again.



Sealed Interface Example

Sealed classes aren't limited to classes. Interfaces can also be sealed.

```
public sealed interface Payment  
    permits CreditCardPayment, CashPayment {  
}
```

Permitted implementations:

```
public final class CreditCardPayment  
    implements Payment {  
}
```

```
public final class CashPayment  
    implements Payment {  
}
```

An unauthorized implementation would fail:

```
public class UpiPayment implements Payment {  
}
```

The compiler reports an error because UpiPayment is not permitted by the sealed Payment interface.

Key Characteristics

Feature	Sealed Classes
Introduced	Java 15 Preview
Second Preview	Java 16
Final	Java 17
Purpose	Restrict inheritance
Keyword	sealed
Permission list	permits
Permitted subclass	final, sealed, or non-sealed
Works with classes	Yes
Works with interfaces	Yes
Benefit	Controlled class hierarchy

In short

Sealed classes in Java 17 provide controlled inheritance by explicitly defining which classes are allowed to extend a class or implement an interface. This makes class hierarchies more predictable, maintainable, and easier for the compiler to reason about.

Java 17 — Strong Encapsulation

Java Version: 17

Year: 2021

Feature: Strong Encapsulation

UniqueClassName: StrongEncapsulation

Definition

Strong encapsulation in Java 17 means that the **internal implementation details of the JDK are strongly protected from unauthorized access**.

Java's module system, introduced in Java 9, started the move toward strong encapsulation. In **Java 17**, the JDK strongly encapsulates most internal APIs by default.

For example, internal packages such as:

```
sun.*  
com.sun.*  
jdk.internal.*
```

are generally no longer accessible through the old reflective-access mechanisms unless specific compatibility mechanisms are used.

Why Strong Encapsulation?

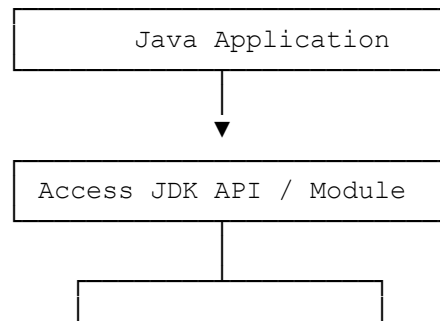
Before strong encapsulation, applications could sometimes access JDK internals using reflection:

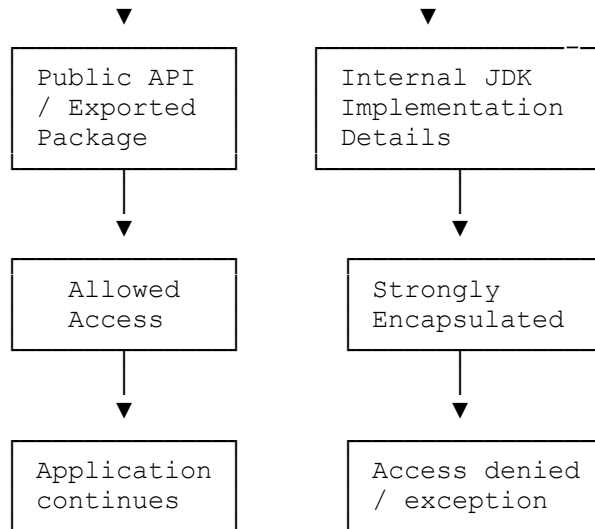
```
Field field = String.class.getDeclaredField("value");  
field.setAccessible(true);
```

Such access relied on implementation details rather than supported Java APIs.

Strong encapsulation encourages developers to use **standard, supported APIs** instead.

Flowchart





Java Code Example

A good way to demonstrate the principle is to compare **supported public APIs** with attempts to access JDK implementation details.

```
public class StrongEncapsulation {  
    public static void main(String[] args) {  
        String text = "Hello Java 17";  
  
        // Supported public API  
        System.out.println("String: " + text);  
  
        // Supported public method  
        System.out.println("Length: " + text.length());  
  
        // Supported public API  
        System.out.println("Uppercase: " + text.toUpperCase());  
    }  
}
```

Output

```
String: Hello Java 17  
Length: 13  
Uppercase: HELLO JAVA 17
```

The application uses the **public API** exposed by String, rather than depending on its internal implementation.

Reflection Example

Consider an attempt to access the internal representation of a String:

```
import java.lang.reflect.Field;

public class StrongEncapsulation {

    public static void main(String[] args) throws Exception {

        String text = "Hello";

        Field field = String.class.getDeclaredField("value");

        field.setAccessible(true);

        Object value = field.get(text);

        System.out.println(value);

    }

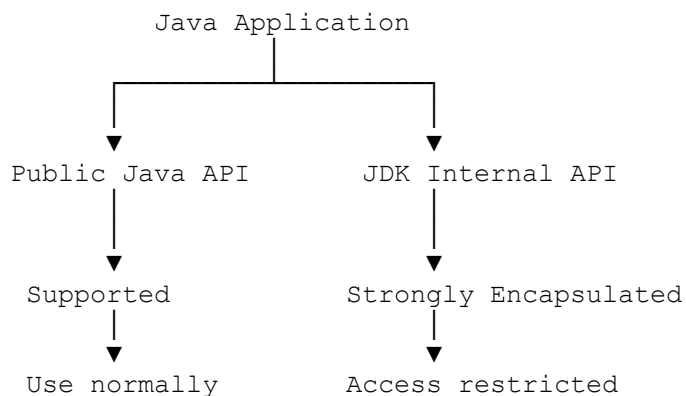
}
```

In Java 17, this kind of reflective access to strongly encapsulated JDK internals can fail with an exception such as:

```
java.lang.reflect.InaccessibleObjectException:
Unable to make field ... accessible:
module java.base does not "opens java.lang" to unnamed module
```

The important point is that `String.value` is an **implementation detail**, not a supported API for application code.

Supported API vs Internal API



For example:

Recommended

```
String text = "Java";
```

```
System.out.println(text.length());
```

Not recommended

```
Field field = String.class.getDeclaredField("value");  
field.setAccessible(true);
```

The second approach depends on the internal representation of String, which can change between JDK implementations or releases.

Key Characteristics

Characteristic	Strong Encapsulation
Major module-system foundation	Java 9
Strongly enforced by default	Java 17
Main mechanism	Java Platform Module System (JPMS)
Protects	JDK internal APIs
Encourages	Standard public APIs
Prevents	Unauthorized access to internal implementation
Reflection on internals	Restricted
Main benefit	Better maintainability and security

In short

Java 17's strong encapsulation prevents ordinary application code from freely accessing JDK internal implementation details, encouraging developers to depend on stable, supported Java APIs instead.